

Making accurate ADC readings on the Arduino

Due to WordPress's abysmal handling of code blocks this blog post is now hosted at <https://majenko.co.uk/blog/>

There are many sensors out there which output a voltage as a function of the supply voltage as their sensed value. Temperature sensors, light sensors, all sorts.

Measuring that voltage, and converting it in to real figures for whatever is being sensed is not actually as simple as you might at first think.

There are many examples on the internet for converting an ADC value into a voltage, but basically it boils down to:

- Divide the ADC value by the ADC maximum value
- Multiply by the supply voltage

And that sounds simple enough, doesn't it?

```
unsigned int ADCValue;
double Voltage;

ADCValue = analogRead(0);
Voltage = (ADCValue / 1024.0) * 5.0;
```

Surely that looks OK, yes? You've got your Arduino plugged into the USB, which is supposedly 5 volts – after all, all the examples on the web just say 5v.

Wrong!

What you have there is a rough approximation. Nothing more.

If you want to make ACCURATE readings you have to know exactly what your supply voltage is.

Measuring the 5V connection on my Arduino while plugged in to the USB is actually reading 5.12V. That makes a big difference to the results of the conversion from ADC to voltage value. And it fluctuates. Sometimes it's 5.12V, sometimes it's 5.14V. so, you really need to know the supply voltage at the time you are doing your ADC reading.

Sounds tricky, yes?

Yes.

However, if you have a known precise voltage you can measure using the ADC, then it is possible to calculate what your supply voltage is. Fortunately, some of the AVR chips used on Arduinos have just such a voltage available, and can be measured with the ADC. Any Arduino based on the 328 or 168 chips has this facility.

I came across this nice piece of code on the [Tinkerit](#) site. It measures this 1.1V reference voltage, and uses the resultant ADV value to work out what the supply voltage must be.

```
long readVcc() {
    long result;
    // Read 1.1V reference against AVcc
    ADMUX = _BV(REFS0) | _BV(MUX3) | _BV(MUX2) | _BV(MUX1);
    delay(2); // Wait for Vref to settle
    ADCSRA |= _BV(ADSC); // Convert
    while (bit_is_set(ADCSRA,ADSC));
    result = ADCL;
    result |= ADCH<<8;
    result = 1125300L / result; // Back-calculate AVcc in mV
    return result;
}

void setup() {
    Serial.begin(9600);
}

void loop() {
    Serial.println( readVcc(), DEC );
    delay(1000);
}
```

Advertisement

Very nice. very elegant. And, more importantly, very useful.

So now, using that, your ADC code could now look like this:

```
unsigned int ADCValue;
double Voltage;
double Vcc;

Vcc = readVcc()/1000.0;
ADCValue = analogRead(0);
Voltage = (ADCValue / 1024.0) * Vcc;
```

And it will be a whole lot more accurate.

Addendum on calibration and accuracy

The internal band-gap, while *nominally* 1.1V can actually be anywhere between 1V and 1.2V. If you want super-accurate readings you may need to adjust the value 1125200 to a more accurate value to represent your band-gap. That value is calculated as the band-gap voltage (in mV) multiplied by 1023. You can do the opposite of the above system and manually measure your Vcc with a DMM, then use that to measure and calculate the band-gap voltage in your chip. Multiply that voltage by 1000 for mV and then by 1023 to get the ADC division value, and Bob's your uncle. From then on, whatever your Vcc voltage does, you can get even more accurate ADC results.